

검색어를 입력하세요.



하네스 엔지니어링 입문

(/book/19559)



0. 들어가며

▼ 1. 하네스 엔지니어링 등장 배경

1.1 프롬프트, 컨텍스트, 하네스 엔지니어링

1.2 에이전트와 하네스 엔지니어링

▼ 2. 하네스 엔지니어링 개념

2.1 하네스 엔지니어링이란?

2.2 Hashimoto의 6단계 도입 여정

2.3 하네스의 4가지 구성요소란?

▼ 3. 하네스 구성요소

3.1 지시문서

3.2 아키텍처 제약

3.3 피드백 루프

3.4 지식 저장소

3.5 가비지 컬렉션

4. 실천구축 (작성중)

5. 사례연구 (작성중)

부록



2.3 하네스의 4가지 구성요소란?



하네스는 네 가지 구성 요소로 이루어집니다. 각각 독립적으로도 효과가 있지만, 네 가지가 함께 작동할 때 온전한 하네스가 됩니다.

구성 요소 한눈에 보기

```
프로젝트 루트/
├── AGENTS.md           ← ① 지시 문서
├── .eslintrc           ← ② 아키텍처 제약
├── tests/              ← ③ 피드백 루프
│   └── ...
└── docs/              ← ④ 지식 저장소
    ├── decisions/
    └── conventions/
```

1) 지시 문서

에이전트에게 규칙을 전달하는 문서

AGENTS.md 또는 CLAUDE.md 같은 파일입니다.

에이전트가 작업을 시작하기 전에 읽는 매뉴얼입니다.

- 코드 스타일, 네이밍 규칙
- 절대 건드리면 안 되는 파일·디렉토리
- PR 작성 방식, 커밋 메시지 형식

사람을 위한 README 처럼
에이전트를 위한 행동 지침입니다.

2) 아키텍처 제약

잘못된 코드를 구조적으로 차단

린터, 타입 검사, 디렉토리 규칙처럼 코드가 저장되거나 병합되기 전에 자동으로 검사하는 장치입니다.

- 허용되지 않는 import 경로 차단
- 코드 스타일 자동 교정
- 특정 패턴 사용 금지

에이전트가 규칙을 몰라도, 잘못된 코드는 통과하지 못합니다.

3) 피드백 루프

에이전트 행동을 실시간으로 교정

에이전트가 작업한 결과가 올바른지 즉시 알려주는 장치입니다. Martin Fowler는 이를 두 가지로 분류합니다.

종류	역할	예시
가이드(Guide)	올바른 방향을 안내	테스트, 예제 코드
센서(Sensor)	잘못된 행동을 감지	CI 실패, 린터 경고

피드백이 빠를수록 에이전트는 더 적은 비용으로 올바른 결과를 냅니다.

4) 지식 저장소

사람의 결정과 맥락을 축적

docs/ 디렉토리에 사람이 내린 결정, 채택한 이유, 포기한 대안을 기록해둡니다.

- 왜 이 라이브러리를 선택했는가
- 이전에 시도했다가 실패한 방법
- 특정 구조를 유지하는 이유

에이전트는 매 세션 백지로 시작합니다.

지식 저장소는 그 공백을 채웁니다.

4가지 요소의 관계

지시 문서

→

에이전트가 "어떻게 할지" 안다

아키텍처 제약

→

잘못된 방향으로 가면 막힌다

피드백 루프

→

결과가 맞는지 즉시 확인한다

지식 저장소

→

왜 이렇게 하는지 이유를 안다

네 요소가 모두 갖춰졌을 때,
에이전트는 새 세션에서도 사람의 규칙과 맥락 안에서 일합니다.

다음 장(3장)에서는 각 구성 요소를 실제 코드와 설정 파일로 직접 만들어봅니다.

마지막 편집일시 : 2026년 4월 12일 9:59 오후

댓글 0

피드백

- 이전글 : 2.2 Hashimoto의 6단계 도입 여정
- 다음글 : 3. 하네스 구성요소

🏠 (/)

🔍

⬆